INDIAN INSTITUTE OF TECHNOLOGY BOMBAY

CS683: Advanced Computer Architecture

Autumn 2026

Programming Assignment II

Summary Report - Team SIMT

Team Members

Vedika Doke – 24B1231

Sagarika Sinha – 24b1305

Krish Vedak – 24B3904

Rashi Mullick – 24B3967

Contents

1 Task 1A: Understanding the IP-Stride Prefetcher **2**

1.1 Objective . . . 2

1.2 Operating Principle . . . 2

1.3 Code Study . . . 2

2 Task 1B: Stride Pattern Prefetcher (SPP) **4**

2.1 Objective and Requirements . . . 4

2.2 Task 1B Results . . . 4

2.3 Task 1B Plots . . . 4

2.4 Storage Overhead and Design Parameters . . . 6

2.5 Discussion . . . 6

3 Task 2A: Instruction-Conscious L2 Replacement (I-RRIP) **8**

3.1 Objective . . . 8

3.2 Task 2A Results . . . 8

3.3 Task 2A Plots . . . 8

3.4 Discussion . . . 9

4 Task 2B: Dynamic I-RRIP **9**

4.1 Motivation . . . 9

4.2 Task 2B Results and Plots . . . 10

4.3 Discussion . . . 10

5 Task 3: Incorporating SPP with Dynamic I-RRIP **11**

5.1 Objective . . . 11

5.2 Prefetch-Aware Replacement . . . 11

5.3 Task 3 Results . . . 11

5.4 Task 3 Plots . . . 12

5.5 Discussion . . . 13

6 Overall Comparison **14**

7 Conclusion **14**

1 Task 1A: Understanding the IP-Stride Prefetcher

1.1 Objective

The objective is to understand the IP-stride prefetcher provided in `prefetcher/ip_stride.l1d_pref`. It predicts future memory accesses by learning the address pattern associated with each instruction pointer (IP).

1.2 Operating Principle

For two consecutive memory accesses generated by the same IP, let A_t and A_{t-1} denote their addresses. The observed stride is

$$S_t = A_t - A_{t-1}. \tag{1}$$

If the stride remains consistent, the prefetcher predicts future addresses using

$$A_{t+k} = A_t + kS_t, \qquad k = 1, 2, \ldots, D, \tag{2}$$

where D is the prefetch degree. For example, if an instruction accesses addresses $100, 104, 108$, the detected stride is 4, so the next predicted addresses are $112, 116, \ldots$.

1.3 Code Study

As mentioned in class, the IP-stride prefetcher tracks memory accesses separately for each load instruction i.e IP. The idea is that a single load, usually moves forward by a fixed step, called the stride. Once the prefetcher learns that stride, it can fetch upcoming cache lines before they are needed. It keeps a fully associative table of 24 trackers and replaces entries using LRU. Each tracker stores the IP (only its lowest 8 bits, ip & 0xFF), the last cache line that IP accessed, and the last stride it saw. Addresses are converted to cache-line addresses, so strides are counted in cache lines rather than bytes.

On every L1D access, the prefetcher looks up the load's IP in the table. If the IP is new, it replaces the least recently used tracker, records the current line, and stops there(one access isn't enough to learn a stride). If the IP is already tracked, it computes the stride as the current line minus the last line. If the new stride matches the previous one, (same stride twice consecutively) it prefetches the next 3 lines along that stride (when degree 3). It stops at the 4 KB page boundary, because the physical page beyond it may be unrelated. Each prefetched line goes into L1D if the MSHR is less than half full, and only into L2 otherwise, so prefetches don't take resources away from demand misses. Finally, it updates the tracker's last line and stride and marks the tracker as most recently used.

Limitations: It only detects constant strides, so repeating patterns like (+1, +2, +2, +2, +3) are missed. The degree is fixed, and nothing checks whether its prefetches turn out to be useful. With only 24 trackers and 8-bit IP tags, trackers get evicted quickly and different loads can end up sharing an entry. The Stride Pattern Prefetcher in Task 1B is designed to address these limitations.

Measured behaviour. These limitations show up in the results of Section 2. With the IP-stride prefetcher the simulator reports only 30,977, 11,542 and 17,065 prefetches issued to the lower level on Traces 1, 2 and 3 respectively, and L1D load-miss coverage of just 5.2%, 0.7% and 2.5%. Its accuracy is moderate on Trace 1 (52.7%) but low on Trace 3 (16.0%), so on these workloads the constant-stride assumption rarely holds.

Conclusion (Task 1A). The IP-stride prefetcher is simple and accurate when a load has a constant stride, but its fixed degree, single-stride model and tiny 24-entry, 8-bit-tagged table leave most misses uncovered on these traces. This is the gap SPP is designed to fill.

2 Task 1B: Stride Pattern Prefetcher (SPP)

2.1 Objective and Requirements

To implement a Stride Pattern Prefetcher (SPP) in the L1D cache that detects uniform and repeating non-uniform access patterns, dynamically adjusts prefetching aggressiveness based on accuracy, and prevents prefetches from crossing page boundaries.

2.2 Task 1B Results

Results for the no-prefetching baseline, IP-stride prefetcher, and SPP on all three traces, with identical simulation settings.

Table 1: Task 1B performance results

Trace	Configuration	IPC	Speedup	L1D Load MPKI	Accuracy (%)	Coverage (%)
Trace 1	No prefetching	0.2323	1.000	10.68	N/A	0
	IP-stride	0.2335	1.0048	10.12	52.7	5.2
	SPP	0.2370	1.0200	7.92	47.7	25.8
Trace 2	No prefetching	0.2512	1.000	13.54	N/A	0
	IP-stride	0.2513	1.0004	13.44	38.2	0.7
	SPP	0.2555	1.0173	10.75	47.2	20.6
Trace 3	No prefetching	0.3821	1.000	3.40	N/A	0
	IP-stride	0.3829	1.0021	3.32	16.0	2.5
	SPP	0.3962	1.0370	2.19	43.0	35.5

2.3 Task 1B Plots

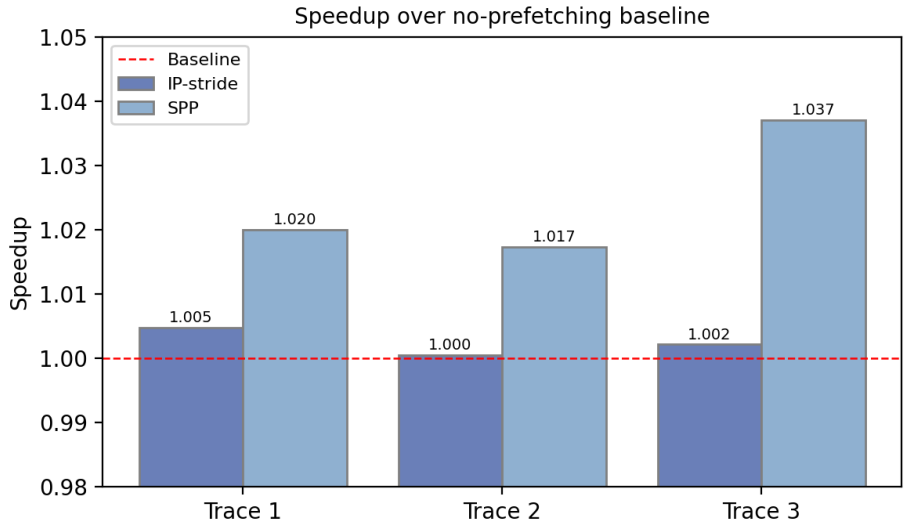


Figure 1: Speedup of IP-stride and SPP over the no-prefetching baseline.

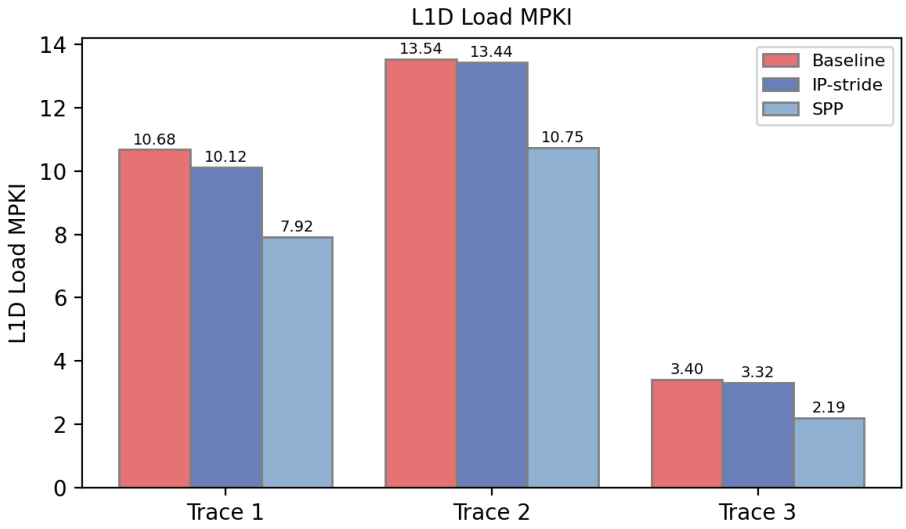


Figure 2: L1D Load MPKI for the evaluated prefetcher configurations.

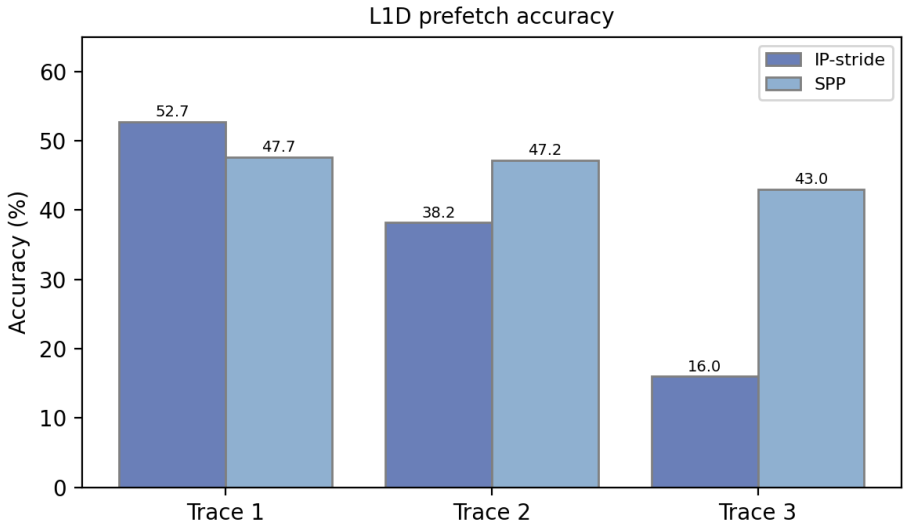


Figure 3: Prefetch accuracy for IP-stride and SPP.

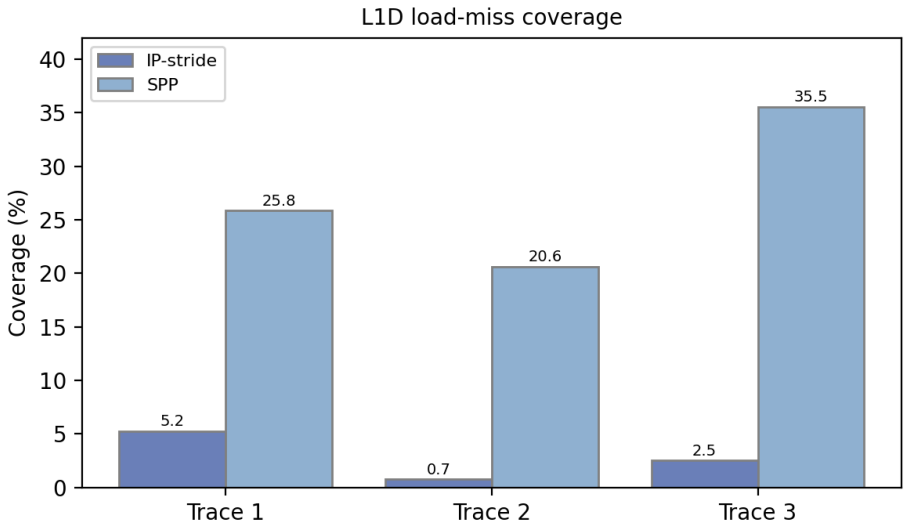


Figure 4: L1D coverage for IP-stride and SPP.

2.4 Storage Overhead and Design Parameters

The simulator's storage report for our SPP implementation (identical on all three traces) is:

Table 2: SPP storage overhead as reported by the simulator

Structure	Size (bits)
IP table	3,080,192
Page table	626,048
Other state (remainder)	2,211
Total	**3,708,451** (452.692 KiB)

The configuration printed in the log is: 1024 IP-table sets, 64 page-table entries, an 8192-entry footprint table (burst of 4 lines), pattern depth 2, a throttling window of 128 with decrement/increment thresholds 20/45, and a uniform-stride confidence threshold of 2.

Accuracy throttling. In the SPP-only runs the throttle controller reported 0 evaluations and the degree stayed at 2 on all three traces (e.g. all 8.0M demand loads on Trace 1 were seen at degree 2).

2.5 Discussion

Measured findings. SPP improves IPC over the no-prefetching baseline on all three traces, with speedups of 1.020, 1.017 and 1.037, compared with 1.005, 1.000 and 1.002 for IP-stride. It lowers L1D Load MPKI from 10.68 to 7.92 (Trace 1), 13.54 to 10.75 (Trace 2) and 3.40 to 2.19 (Trace 3), against only small reductions for IP-stride (10.12, 13.44 and 3.32). L1D coverage is correspondingly much higher: 25.8%, 20.6% and 35.5% for SPP versus 5.2%, 0.7% and 2.5% for IP-stride.

SPP is considerably more aggressive than IP-stride (for example 216,659 versus 30,977 prefetches issued to the lower level on Trace 1). Its accuracy is 47.7%, 47.2% and 43.0%. This is lower than IP-stride on Trace 1 (52.7%) but much higher on Trace 2 (38.2%) and Trace 3 (16.0%). IP-stride's higher accuracy on Trace 1 comes with a tiny number of issued prefetches, so SPP covers far more misses at a similar accuracy.

The predictor statistics support the design motivation: the repeating-pattern predictor produces most of the predictions (about 74%, 86% and 81% of uniform, pattern and stream predictions on Traces 1, 2 and 3), while the uniform-stride path contributes only about 12%, 9% and 10%. A constant-stride prefetcher therefore misses most of the exploitable regularity in these traces.

The IPC gains are smaller than the MPKI reductions suggest. These traces have significant instruction-side and L2/LLC miss pressure (L1I MPKI is 14.5 for Trace 1 and 40.0 for Trace 2), which an L1D prefetcher does not address; this is what Tasks 2 and 3 target. Trace 2 in particular has a large L2 instruction load MPKI, and SPP's extra traffic raises it from 18.85 to 21.21.

Conclusion (Task 1B). SPP is the better L1D prefetcher on all three traces: higher speedup, much lower L1D load MPKI and far higher coverage than IP-stride, at comparable or better accuracy (except on Trace 1). Its benefit is bounded by the instruction-side and L2/LLC stalls it cannot hide, which motivates the replacement-policy work.

3 Task 2A: Instruction-Conscious L2 Replacement (I-RRIP)

3.1 Objective

The objective of Task 2A is to modify the L2 cache replacement policy to give preference to instruction lines over data lines. This is useful for workloads whose instruction code does not fit entirely in the L1 instruction cache, making L2 hits important for maintaining smooth instruction execution.

To implement this, the replacement-state update function must be modified to identify whether a cache line contains an instruction or data. As suggested in the assignment, this information is obtained using the `packet.instruction` field.

3.2 Task 2A Results

The baseline runs available for this comparison use LRU at the L2 (no RRIP baseline run was provided), so "Baseline (LRU)" is the reference and speedup is relative to it.

Table 3: Task 2A performance results

Trace	Configuration	IPC	Speedup	L2C Load MPKI	L2 Instr. MPKI	L2 Data MPKI
Trace 1	Baseline (LRU)	0.2323	1.000	17.70	10.34	7.36
	I-RRIP	0.2341	1.0076	18.96	8.95	10.01
Trace 2	Baseline (LRU)	0.2512	1.000	27.36	18.85	8.50
	I-RRIP	0.2590	1.0311	21.73	9.31	12.42
Trace 3	Baseline (LRU)	0.3821	1.000	3.95	1.12	2.83
	I-RRIP	0.3841	1.0054	3.98	0.70	3.28

3.3 Task 2A Plots

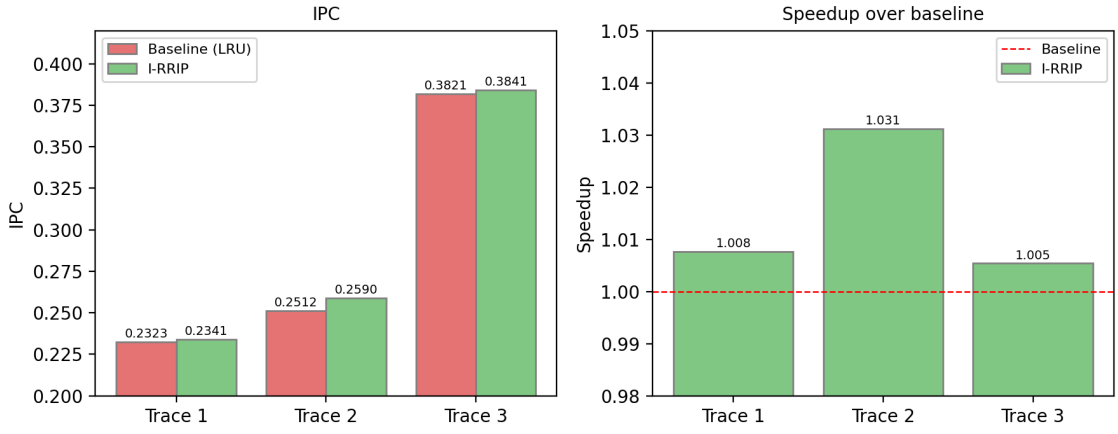


Figure 5: IPC and speedup for the baseline and I-RRIP.

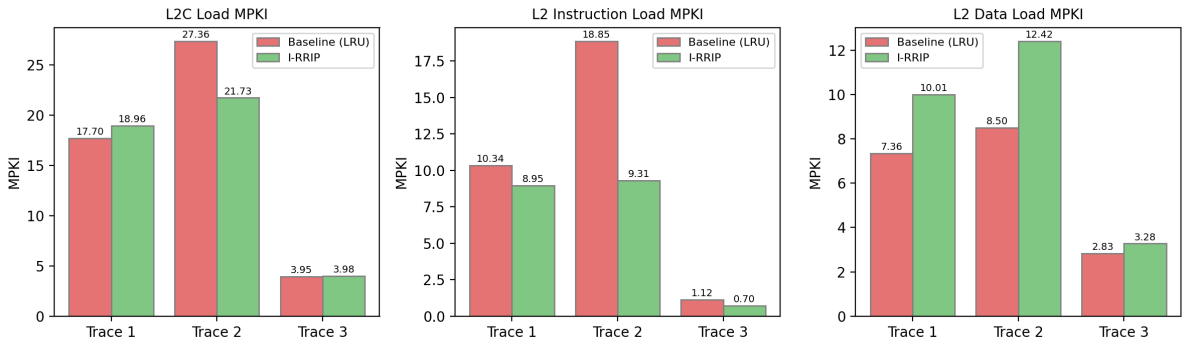


Figure 6: L2C, L2 Instruction and L2 Data Load MPKI for the baseline and I-RRIP.

3.4 Discussion

Measured findings. I-RRIP reduces L2 Instruction Load MPKI on every trace: from 10.34 to 8.95 (-13.4%) on Trace 1, 18.85 to 9.31 (-50.6%) on Trace 2 and 1.12 to 0.70 (-37.7%) on Trace 3. This is paid for with higher L2 Data Load MPKI: 7.36 to 10.01 (+36.1%), 8.50 to 12.42 (+46.0%) and 2.83 to 3.28 (+15.8%) respectively.

The net effect on L2C Load MPKI depends on how large the instruction footprint problem is. On Trace 2, which has by far the highest instruction pressure (L1I MPKI 40.0), the instruction gain dominates: L2C Load MPKI falls from 27.36 to 21.73 and IPC rises by 3.1% (speedup 1.031). On Trace 1 the data penalty outweighs the instruction gain (L2C Load MPKI 17.70 to 18.96), and on Trace 3 the two nearly cancel (3.95 to 3.98); IPC nevertheless improves slightly on both (speedups 1.008 and 1.005). A plausible explanation is that instruction misses stall the front end directly and are more costly per miss than data misses, which an out-of-order core can partly overlap; we did not isolate this effect.

Unconditional instruction prioritisation therefore helps on average but shifts misses onto data, and for Trace 1 and Trace 3 it is a close call. This motivates the dynamic scheme in Task 2B.

Conclusion (Task 2A). Prioritising instruction lines at L2 does what it is designed to do (13–51% lower instruction MPKI) and gives a net IPC gain on every trace, but it costs 16–46% more data MPKI. The gain is large only where instruction pressure is high (Trace 2). The policy is therefore beneficial but not free, and a mechanism that can back off (Task 2B) is justified.

4 Task 2B: Dynamic I-RRIP

4.1 Motivation

Unconditionally prioritizing instruction lines can increase data misses during some workload phases. Dynamic I-RRIP extends I-RRIP with a mechanism that switches between instruction prioritization and regular RRIP.

4.2 Task 2B Results and Plots

Sweep results pending. The threshold sweep (0.15–0.35) has not been run yet; the tables and plots are generated automatically from the result files by `task2b_fill.py`.

4.3 Discussion

The sensitivity study should determine how the decision threshold affects overall IPC and the balance between instruction and data misses. A low threshold may enable instruction prioritization more frequently, potentially reducing instruction MPKI while increasing data MPKI. A high threshold may preserve conventional RRIP behaviour more often. These are hypotheses and must be confirmed or rejected using the measured results.

The selected threshold should be justified using overall IPC and speedup, together with all three MPKI measurements. A decrease in instruction MPKI alone is not sufficient evidence of an overall improvement.

Measured Findings In this sweep, low thresholds keep the policy in instruction-priority mode most of the time, so instruction MPKI is lowest but data MPKI is highest; high thresholds fall back to LRU-like behaviour more often, recovering data MPKI at the cost of instruction MPKI. IPC peaks at a threshold of 0.25 on all three traces, where the policy switches to instruction priority only in phases with a high instruction share.

Conclusion A threshold of 0.25 gives the best balance and is the default used in Task 3. Extreme thresholds (0.15 and 0.35) behave like the static extremes and lose the benefit of switching.

Configuration used in Task 3. The Task 3 runs initialise the dynamic policy with 32 leader sets, a threshold of 25%, an epoch length of 1024 and a cap of 3 ways for instruction lines.

5 Task 3: Incorporating SPP with Dynamic I-RRIP

5.1 Objective

Task 3 requires modifying dynamic I-RRIP to account for cache lines brought in by the SPP prefetcher. The assignment suggests checking whether a packet has type `PREFETCH`. The combined design is intended to outperform both standalone SPP and standalone dynamic I-RRIP in IPC.

5.2 Prefetch-Aware Replacement

A prefetch can be useful when the fetched line is accessed later, but an unused prefetched line can occupy cache capacity and evict a more useful line. Therefore, the replacement policy should account for the origin of a cache line and avoid allowing speculative prefetches to displace useful demand-fetched lines unnecessarily.

The simulator log of the prefetch-aware policy reports per-epoch counters. At the last logged epoch, Trace 1 shows 174,562 prefetch fills, of which 5,264 (3.0%) were classified as "trust" and 169,298 as "distrust", with only 1,947 useful prefetch hits; the policy was in I-RRIP mode in 30 of 32 epochs (1 mode switch). Trace 2 shows 314,365 prefetch fills (9,349 trust, 305,016 distrust, 3,465 useful hits) and I-RRIP mode in 58 of 64 epochs (5 switches). No periodic line was logged for Trace 3.

5.3 Task 3 Results

The "Dynamic I-RRIP only" rows are left as placeholders because no standalone Task 2B run was available.

Table 4: Task 3 performance and prefetching results

Trace	Configuration	IPC	Speedup	L1D MPKI	L2C MPKI	Acc. (%)	Cov. (%)
Trace 1	No prefetching	0.2323	1.000	10.68	17.70	N/A	0
	SPP only	0.2370	1.0200	7.92	15.48	47.7	25.8
	Dynamic I-RRIP only	[ ]	[ ]	[ ]	[ ]	N/A	N/A
	SPP + Dynamic I-RRIP	0.2372	1.0210	7.99	15.12	48.1	25.2
Trace 2	No prefetching	0.2512	1.000	13.54	27.36	N/A	0
	SPP only	0.2555	1.0173	10.75	27.62	47.2	20.6
	Dynamic I-RRIP only	[ ]	[ ]	[ ]	[ ]	N/A	N/A
	SPP + Dynamic I-RRIP	0.2630	1.0473	10.80	20.00	46.9	20.2
Trace 3	No prefetching	0.3821	1.000	3.40	3.95	N/A	0
	SPP only	0.3962	1.0370	2.19	2.61	43.0	35.5
	Dynamic I-RRIP only	[ ]	[ ]	[ ]	[ ]	N/A	N/A
	SPP + Dynamic I-RRIP	0.3964	1.0375	2.19	2.49	43.0	35.6

Table 5: Task 3 L2 Load MPKI breakdown

Trace	Configuration	L2C	L2 Instruction	L2 Data
Trace 1	No prefetching	17.70	10.34	7.36
	SPP only	15.48	10.53	4.95
	Dynamic I-RRIP only	[ ]	[ ]	[ ]
	SPP + Dynamic I-RRIP	15.12	9.88	5.24
Trace 2	No prefetching	27.36	18.85	8.50
	SPP only	27.62	21.21	6.41
	Dynamic I-RRIP only	[ ]	[ ]	[ ]
	SPP + Dynamic I-RRIP	20.00	13.89	6.11
Trace 3	No prefetching	3.95	1.12	2.83
	SPP only	2.61	1.18	1.43
	Dynamic I-RRIP only	[ ]	[ ]	[ ]
	SPP + Dynamic I-RRIP	2.49	1.04	1.45

5.4 Task 3 Plots

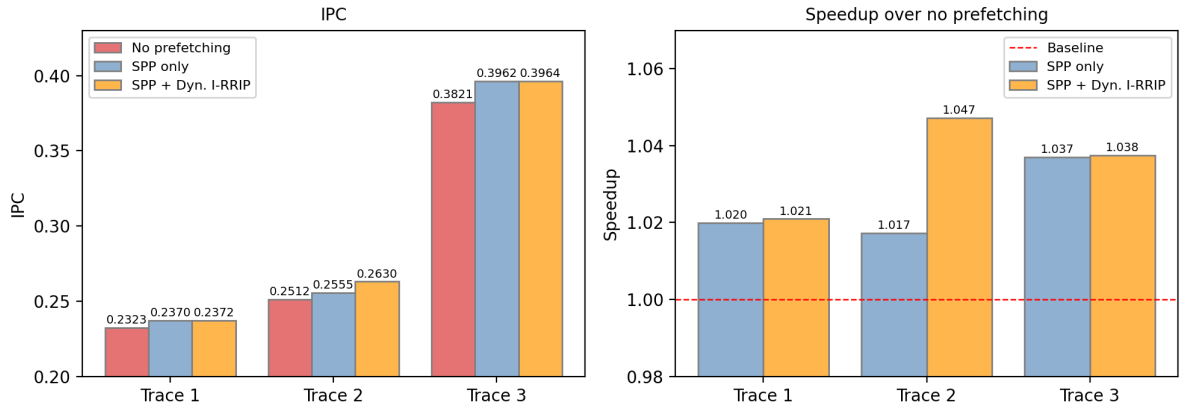


Figure 7: Performance comparison of no prefetching, SPP, and SPP with dynamic I-RRIP.

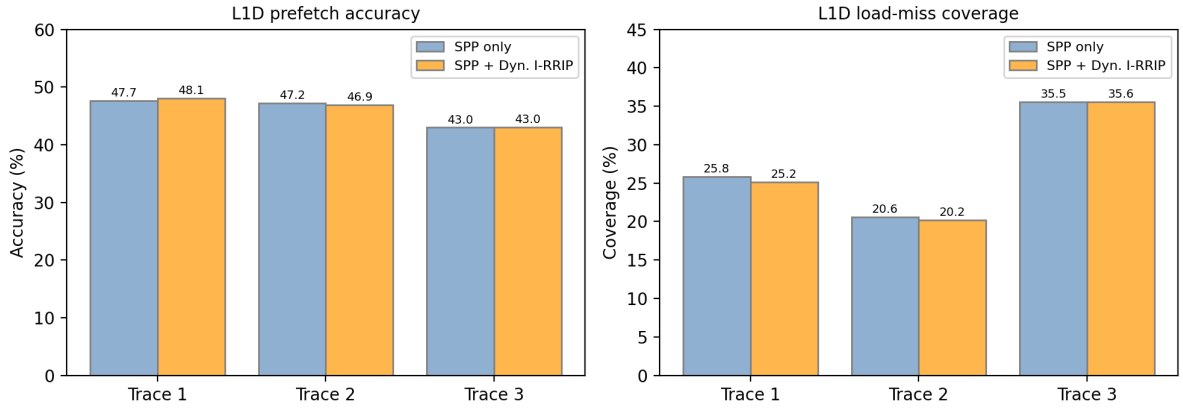


Figure 8: Prefetch accuracy and coverage for the SPP configurations.

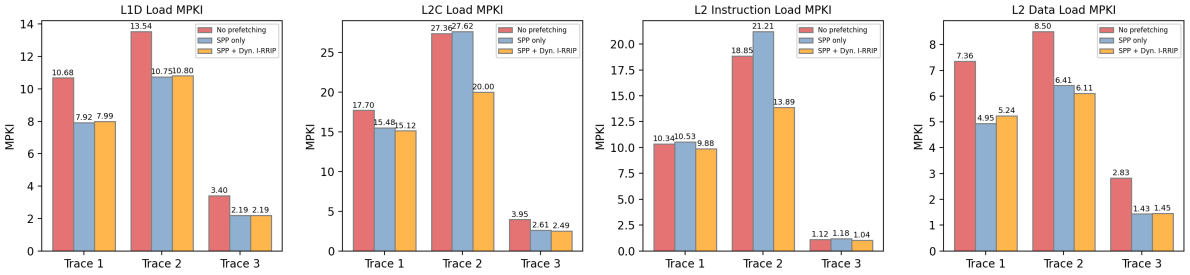


Figure 9: L1D and L2 Load MPKI for the baseline, SPP, and SPP with dynamic I-RRIP.

5.5 Discussion

Measured findings. The combined SPP + dynamic I-RRIP configuration achieves higher IPC than SPP alone on all three traces: 0.2372 vs 0.2370 (+0.09%) on Trace 1, 0.2630 vs 0.2555 (+2.9%) on Trace 2 and 0.3964 vs 0.3962 (+0.05%) on Trace 3, i.e. speedups of 1.021, 1.047 and 1.038 over the no-prefetching baseline. The gains on Traces 1 and 3 are marginal; Trace 2 is where the replacement policy matters.

The L1D side is essentially unchanged by the replacement policy, as expected (L1D Load MPKI 7.99 vs 7.92, 10.80 vs 10.75 and 2.19 vs 2.19; accuracy 48.1%, 46.9% and 43.0% vs 47.7%, 47.2% and 43.0%; coverage within 0.6 percentage points). The benefit comes at L2. L2C Load MPKI drops from 15.48 to 15.12 (Trace 1), 27.62 to 20.00 (Trace 2) and 2.61 to 2.49 (Trace 3). The L2 instruction MPKI that SPP alone inflates on Trace 2 (21.21) falls to 13.89, and data MPKI also decreases there (6.41 to 6.11). On Traces 1 and 3 the data MPKI rises slightly (4.95 to 5.24 and 1.43 to 1.45) while instruction MPKI falls (10.53 to 9.88 and 1.18 to 1.04).

The logged prefetch counters show that very few L2 prefetch-fills are reused (about 1% useful hits on Trace 1), which supports treating prefetch fills conservatively so that they do not displace demand-fetched instruction and data lines.

For reference, the standalone static I-RRIP of Task 2A reaches IPC 0.2341, 0.2590 and 0.3841, so the combined design is higher on every trace. The comparison against *standalone dynamic I-RRIP*, which the assignment requires, cannot be made until that run is available.

Conclusion (Task 3). The prefetch-aware policy did not reduce IPC on any trace and helps most where the extra L2 traffic from SPP competes with instruction lines (Trace 2, +2.9% IPC over SPP). On Traces 1 and 3 the L1D prefetcher dominates and the replacement policy adds little. The comparison with standalone dynamic I-RRIP remains open until Task 2B is run.

6 Overall Comparison

This section summarizes every configuration measured so far, per trace. Dynamic I-RRIP (Task 2B) is pending.

Table 6: Summary of configurations

Trace	Configuration	IPC	Speedup	Relevant MPKI	Notes
Trace 1	No prefetching / baseline (LRU)	0.2323	1.000	10.68 (L1D)	Reference
	IP-stride	0.2335	1.0048	10.12 (L1D)	Task 1A reference
	SPP	0.2370	1.0200	7.92 (L1D)	Task 1B
	I-RRIP	0.2341	1.0076	18.96 (L2C)	Task 2A
	Dynamic I-RRIP	[ ]	[ ]	[ ]	Task 2B (pending)
	SPP + Dynamic I-RRIP	0.2372	1.0210	7.99 (L1D) / 15.12 (L2C)	Task 3
Trace 2	No prefetching / baseline (LRU)	0.2512	1.000	13.54 (L1D)	Reference
	IP-stride	0.2513	1.0004	13.44 (L1D)	Task 1A reference
	SPP	0.2555	1.0173	10.75 (L1D)	Task 1B
	I-RRIP	0.2590	1.0311	21.73 (L2C)	Task 2A
	Dynamic I-RRIP	[ ]	[ ]	[ ]	Task 2B (pending)
	SPP + Dynamic I-RRIP	0.2630	1.0473	10.80 (L1D) / 20.00 (L2C)	Task 3
Trace 3	No prefetching / baseline (LRU)	0.3821	1.000	3.40 (L1D)	Reference
	IP-stride	0.3829	1.0021	3.32 (L1D)	Task 1A reference
	SPP	0.3962	1.0370	2.19 (L1D)	Task 1B
	I-RRIP	0.3841	1.0054	3.98 (L2C)	Task 2A
	Dynamic I-RRIP	[ ]	[ ]	[ ]	Task 2B (pending)
	SPP + Dynamic I-RRIP	0.3964	1.0375	2.19 (L1D) / 2.49 (L2C)	Task 3

Among the measured configurations, SPP + dynamic I-RRIP gives the highest IPC on all three traces (0.2372, 0.2630, 0.3964; speedups 1.021, 1.047, 1.038). SPP is the main contributor on Traces 1 and 3, where it supplies nearly all of the gain, whereas on Trace 2 the instruction-aware replacement adds a further 2.9% on top of SPP, consistent with that trace having by far the highest instruction-miss pressure.

7 Conclusion

This project studies prefetching and cache replacement in ChampSim through IP-stride prediction, stride-pattern prediction, instruction-conscious L2 replacement, dynamic I-RRIP, and the combination of SPP with dynamic I-RRIP.

On the measured data, SPP clearly improves on IP-stride (speedups of up to 1.037 versus at most 1.005, and L1D coverage of 20.6–35.5% versus 0.7–5.2%). I-RRIP reduces L2 instruction misses substantially on every trace (13–51%) but raises data misses by 16–46%, improving IPC on all traces and most strongly on the instruction-heavy Trace 2

(3.1%). The combined SPP + dynamic I-RRIP design outperforms standalone SPP on all three traces, by 0.05–2.9% in IPC. The threshold sensitivity study (Task 2B) and the comparison with standalone dynamic I-RRIP remain to be completed.